

BANCO DE DADOS AVANÇADO

Gerenciamento de Transações

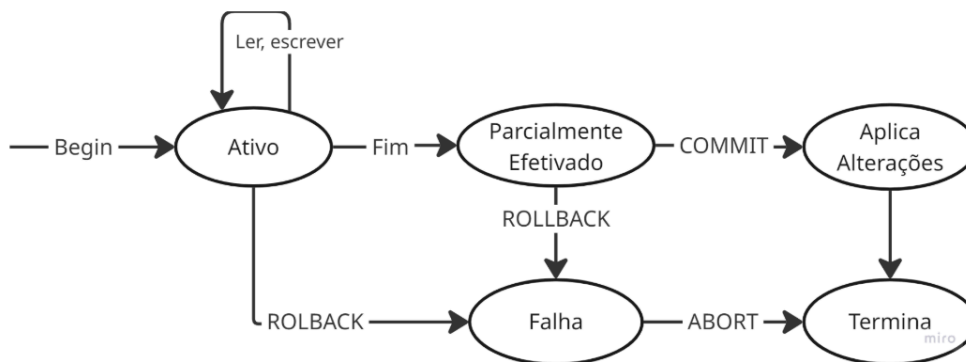
Fundamentos

Uma transação é uma **unidade de trabalho** que possui um ciclo de vida, na qual é executada uma **instrução** ou um **conjunto de transações DML**.

Comandos TCL (Transaction Control Language)

- **BEGIN:** inicia a transação
- **COMMIT:** confirma as alterações
- **ROLLBACK:** desfaz todas as alterações (somente se o commit não tiver sido feito)

Ciclo de Vida



1. **Início:** Primeira **Query** (comando SQL) executada.

- **BEGIN** ou **START**

2. **Fim:** **COMMIT** ou **ROLLBACK**

- DDL
- DMCL
- Lost Connection
- Falha no Sistema (ROLLBACK automático)

Propriedades ACID

Devem seguir as propriedades **ACID**, regras impostas pelo **Controle de Concorrência e Recuperação** do **SGBD**:

- **Atomicidade:** deve ser completa ou abortada.
- **Consistência:** deve ser consistente antes e depois da transação.
- **Isolamento:** transações simultâneas não devem interferir entre si.
- **Durabilidade:** alterações bem-sucedidas, mesmo que o sistema falhe, os dados devem ser salvos permanentemente e não devem ser perdidos.

Plano de Execução (Schedules)

Uma **Schedule** é a **ordem de execução** das operações de **leitura (R)** e de **escrita (W)** de várias transações.

- **Execução Serial:** transações executadas uma após a outra, sem intercalação e interrupções entre elas.

Anomalias/Conflitos

Ocorrendo em diferentes transações, as **anomalias/conflitos** costumam acontecer quando: o isolamento não é bem gerenciado, acessam o mesmo dado e qualquer transação realiza a escrita.

- **Lost Update:** Duas transações acessam o mesmo item e têm suas operações intercaladas.
 - Duas transações leem o mesmo dado simultaneamente e tentam atualizá-lo com base no valor lido.
 - A T1 finaliza, depois tudo começa em T2 (R1 → W1 → COMMIT T1 → R2 → W2 → COMMIT T2)
- **Dirty Read (Uncommitted):** Transação atualiza o valor no banco e depois falha.

- **Phantom Read:** Transação executa a mesma consulta de intervalo duas vezes e recebe um conjunto diferente de linhas, pois outra transação inseriu ou excluiu registros.
- **Non-Repeatable Read:** Transação lê o mesmo dado duas vezes e obtém valores diferentes por causa de outra transação ter modificado e confirmado (commit) esse dado no intervalo entre as leituras.

Locks

- Garantem a serialização (execução em sequência);
- Utilizada para evitar anomalias/conflitos;
- Variável associada a um item no banco de dados;
- Tabela separada;
- Bloqueia a **leitura (R)** e/ou a **escrita (W)** visualizadas em outras transações.

Tipos

Binário

Garante o acesso a um item para uma única transação. Quando uma transação bloqueia um dado, nenhuma pode acessá-lo.

- **Lock:** lock_item(x)
- **Unlock:** unlock_item(x)

Binário(item x (qual item participa), Lock, transação (T), Fila (Q))

Funcionamento

- Se $L(X) = 0 \rightarrow$ qualquer T acessa
- Se $L(X) = 1 \rightarrow$ apenas o T que executou o lock acessa

T tenta Lock(X)

- Se $L = 1 \rightarrow Q(x)$
- Se $L = 0 \rightarrow L(x) \leftarrow 1$

T executa Unlock(x)

- $L(x) \leftarrow 0$
- Executa T na fila

Regras

T deve executar:

- $L(x)$ antes de $R(x)$ e $W(x) \leftarrow$ Lock no item da transação (T)
- $U(x)$ depois de $R(x)$ ou $W(x) \leftarrow$ Após as operações com o item, deve-se dar Unlock no item.

T não pode:

- $L(x)$ se $L(x) = 1 \leftarrow$ Se o Lock estiver bloqueado, coloca a transação em fila de espera.
- $U(x)$ se não foi executado $L(x)$.

Multimodo

Diferencia a **intenção da transação**, operando com:

- **Bloqueio de Leitura (Shared Lock):** múltiplas transações podem adquirir o bloqueio simultaneamente do mesmo dado.
- **Bloqueio de Escrita (Exclusive Lock):** para alterar o dado, exige exclusividade total. O banco só permite se o dado estiver destravado.
- **Destravado (Unlock)**

Regras

T deve executar:

- Read Lock (RL) ou Write Lock (WL) antes de $R(x) \rightarrow$ leitura
- Write Lock (WL) antes de $W(x) \rightarrow$ escrita
- $U(x)$ depois de $R(x)$ e/ou $W(x)$.

Upgrade: Se T for única que possui RL(x), então pode executar WL(x)

Downgrade: Uma transação pode executar WL(x) e depois RL(x).

A **Write Lock** garante que apenas aquela transação está fazendo WL(x)

2PL (Two-Phase Locking)

É um protocolo de segurança do SGBD que divide a transação em duas fases:

1. **Expansão:** adquire lock, não libera nenhum (nenhum unlock).
2. **Contratação:** libera locks; não pode mais adquirir nenhum lock novo (nenhum lock).

Problemas com Locks

Deadlock

- T1 bloqueia o item A e espera o item B.
- T2 bloqueia B e espera o item A.
- As duas ficam travadas infinitamente.

Controle de Deadlocks

- **TimeStamp:** cada T recebe um TS(T) único.
 - **Wait-Die:** se $TS(T1) < TS(T2)$, a T2 é abortada e reinicia.
 - **Wait:** se $TS(T1) < TS(T2)$, a T2 é abortada e reinicia com o mesmo timestamp.
 - **No waiting:** transação aborta depois de um tempo pré-definido.
 - **Cautions wait:** T1 tenta bloquear um item x.

Starvation (Inanição)

Uma transação espera indefinitivamente por um recurso enquanto outras transações continuam passando na frente dela.

Níveis de Isolamento

É um padrão ANSI SQL que define até que ponto uma transação pode causar anomalias em outra e quais anomalias são permitidas para ganhar desempenho.

- 
1. **Read Uncommitted:** lê valores não commitados.
 2. **Read Committed (Padrão):** lê somente valores commitados.
 3. **Repeatable:**
 - Snapshots que criam versões separadas do banco de dados.
 - Boa para relatórios.
 - Uma única versão consistente sem alterações de transação.
 4. **Serializable (Mais rigoroso):**
 - Se o banco não conseguir garantir **transações cereais**, ele aborta.
 - Mesmo que apenas **POSSA** causar conflitos, o banco aborta.
 - Garante que **não haja anomalias de qualquer maneira**.

MVCC (Multiversion Concurrency Control)

- Cria versões estáveis de um dado (registro/linha).
- **Escritores** não bloqueiam **leitores**.
- **Leitores** não bloqueiam **escritores**.
- Todo **escritor(writer)** cria uma nova versão do dado.
- **Para leitores:** visualizam o dado conforme o nível de isolamento.

Tópicos Extras

Como funciona o UPDATE?

No PostgreSQL, o UPDATE não apaga o dado na hora, ele cria uma **nova versão da tupla (linha/registro)**.

Metadados Importantes

- **xmin:** guarda o ID da transação que inseriu ou criou aquela versão do registro/tupla.
- **xmax:** guarda o ID da transação que deletou ou atualizou o registro/tupla.
- **ctid:** é o endereço físico (bloco e offset) do registro/tupla no disco.

Limpeza (Vacuum)

- Quando uma versão importante da tupla/registro não é mais vista por nenhuma transação ativa, ela vira uma **Dead Tuple** (tupla/registro morto).
- O comando **VACUUM** normal marca o espaço dessas dead tuples como **reutilizável internamente** pelo banco, **mas não devolve o espaço para o Sistema Operacional**.
- O comando **VACUUM FULL** reescreve toda a tabela, compactando os dados fisicamente e devolvendo o espaço livre para o Sistema Operacional.

Otimização de Consultas e Segurança

Armazenamento

No **disco**, o banco armazena as tuplas/registros em **blocos/páginas de memória física**.

- **Arquivo Sequencial (Sorted File):** Os registros são mantidos fisicamente **ordenados** por um campo específico, como **id** ou **data**. Possui **alto custo de escrita e leitura**.
- **Arquivo Aleatório (Heap File):** Os registros são gravados em qualquer lugar do disco onde houver espaço livre. Possui **baixo custo de escrita e alto custo de leitura**.

Index

Um índice (index) é uma estrutura de dados utilizada para acelerar consultas e recuperação de linhas em uma tabela. Sua estrutura é baseada em Hashing, Árvore B e B+.

Entradas do Índice

- **K**: Key/Chave de Busca.
- **RiD**: Posição do Disco, composta pelo par (**Bloco**, **Posição**).
- **PK**: Usada em índices secundários de tabelas organizadas por chave, onde se armazena o valor da PK, em vez de RiD, para localizar o registro no índice primário.
- **<K, RiD>**: Chaves únicas para registro.
- **<K, RiD list>**: Uma chave para um conjunto de registros.

Separação Física

- **Tabela Original (Heap)**: guarda todas as colunas do registro (id, nome, email, status, valor etc.), onde os dados são inseridos onde houver espaço nos blocos de memória do disco aleatoriamente.
- **Tabela do Índice**: estrutura de dados auxiliar otimizada que guarda as **chaves de busca** e a localização do dado na tabela original. O seu atributo de chave de busca não é PK nem FK.

Tipos de Index

- **Index Ordenado (Clustered/Denso ou Esparso)**: define a própria ordem física em que os registros/tuplas estão gravados no disco.
- **Index Primário**: baseado em um campo-chave.
 - **Ordenado e Denso**: para cada **K**, tenho uma **RiD**.
- **Index Secundário**: baseado em qualquer campo ou valor.
 - **Esparso**: para cada grupo de **K**, tenho uma **RiD**.

Quando usar?

- Filtros com alta seletividade (Index Scan);
- Junção de tabelas (JOIN);
- Ordenação (ORDER BY) e agrupamento (GROUP BY).

Quando não usar?

- **Consultas com baixa seletividade:** tabelas pequenas ou retornos grandes.
- Tabelas com volume massivo de INSERT, UPDATE e DELETE;
- Colunas com baixa cardinalidade.

Os índices aumentam a velocidade de **READ**, mas diminuem a velocidade de **WRITE**.

1. **Armazenamento**
2. **Leitura**
3. **Escrita** (INSERT, UPDATE, DELETE)



Formas de Índice

1. Índice Simples:

- Criado sobre uma única coluna da tabela.
- Use quando as colunas costumam filtrar, agrupar ou ordenar de forma isolada por uma coluna específica, como id.
- **Exemplo:** CREATE INDEX idx_vendas_cliente ON vendas (cliente_id).

2. Índice Composto (Multicoluna):

- Criado sobre duas ou mais colunas.
- A ordem das colunas altera a organização física dos dados.
- **Exemplo:** CREATE INDEX idx_vendas_cliente ON vendas (cliente_id, data_venda DESC).

3. Índice Parcial:

- Em vez de indexar todas as linhas, criam-se entradas apenas para as **linhas que satisfazem uma condição (WHERE)**.
- **Exemplo:** CREATE INDEX idx_vendas_pendentes ON vendas_teste (cliente_id) WHERE status = 'PENDENTE'

Tipos de Chave

- **Primary Key:** A chave candidata escolhida para identificar a tabela (gera índice primário automático).
- **Candidata:** Super-chave mínima, onde nenhuma coluna pode ser removida sem perder a unicidade.
- **Super-chave ou Composta:** Combinação de colunas que garante unicidade.

Processamento e Otimização de Consultas

O banco de dados passa por etapas para encontrar o **caminho mais barato** com **menos custo** para as **operações de leitura e escrita** e para a **CPU**.

Lei de Moore

O **poder de processamento (CPU)** cresce mais rápido que a velocidade de **leitura de discos** (duplica a cada 2 anos) e os dados crescem exponencialmente.

O volume gerado de dados e a necessidade de armazenamento superam a velocidade de evolução do processamento de operações de leitura e escrita em disco/rede. Por isso, o **otimizador busca reduzir a quantidade de leitura de blocos em disco**.

Execução de Consultas

1. **Consulta de Alto Nível (Análise Léxica e Sintática):** Verifica se a consulta (texto declarativo) está escrita corretamente e se os objetos existem.
 - **Análise Léxica:** verifica se os comandos SQL estão escritos corretamente.
 - **Análise Sintática:** verifica se a sintaxe está correta.
2. **Otimização (Planner/Otimizador):** Avalia estatísticas e decide o plano de execução (Scans e JOINS).
3. **Código de Consulta:** Transforma o plano em instruções de baixo nível.
4. **Execução de Código:** Executa o código e retorna os resultados.

Métricas de Escolha

Conceito	Definição	Exemplo
Seletividade	Capacidade de isolamento de poucas linhas em um filtro. Filtros muito seletivos retornam um resultado pequeno de uma tabela.	Retornar 1 registro em 2 milhões representa alta seletividade . Retornar 80% da tabela representa baixa seletividade .
Cardinalidade	Quantidade de valores distintos em uma coluna. Colunas como CPF ou ID têm cardinalidade máxima.	Uma coluna codigo_transacao com quase 2 milhões de valores diferentes possui alta cardinalidade.

Tipos de Scan

O **Otimizador (Planner)** calcula a **seletividade da busca** e a **cardinalidade dos dados** para escolher o tipo de **Scan (varredura)** para **minimizar o custo de Input/Output** (leitura de blocos no disco).

Tipos	Características	Tipo de Acesso ao Disco	Seletividade	Quando escolher?
Sequential Scan	Percorre todas as linhas. Não possui índice útil . Retorna muitas linhas. Tê-la não significa que o plano é ruim.	Leitura sequencial em uma fila contínua	Baixa	Tabelas pequenas ou filtros com baixa seletividade
Index Scan	Lê o índice (K, RiD) e vai direto ao bloco do registro (Heap)	Aleatória, saltando para blocos diferentes a cada linha	Altíssima	Poucas linhas e filtros altamente seletivos.
Bitmap Index Scan	Varre o índice e monta um mapa de índice com os endereços de blocos necessários.	Leitura do índice	Intermediária	Quando há mais linhas do que o Index Scan tolera de forma eficiente.
Bitmap Heap Scan	Lê os blocos da tabela original marcados no Bitmap em ordem física crescente.	Sequencial otimizada, percorrendo apenas os blocos úteis em ordem	Intermediária	Busca em volumes médios de dados minimizando os saltos nos discos.

Consultas

Uma **consulta** é qualquer pergunta feita ao computador em qualquer contexto.

Consultas Declarativas

Como o **SQL** é uma linguagem declarativa, declaramos **o que queremos do banco de dados**, e não como fazer (Python).

Processamento de Consultas Declarativas

Blocos de consulta ou simples com uma única expressão, com SELECTs separados e sem aninhamento (SELECT dentro de SELECT).

Tipos de Consultas

1. **Pesquisa Linear:** leitura da tabela bloco por bloco.
2. **Pesquisa Binária:** A
 - É aplicada quando o arquivo de dados da tabela está fisicamente ordenado por um campo específico e sem um índice.
 - O SGBD realiza divisões sucessivas ao meio dos blocos até encontrar o registro.
 - Deve estar ordenada.
3. **Busca por Índice Primário:** busca realizada no índice criado automaticamente sobre a PK da tabela.
4. **Busca por Chave Hash:** aplica uma função matemática de hashing sobre o valor pesquisado para calcular o endereço da memória/disco do registro/tupla.
5. **Busca por Índice Secundário:** índices criados manualmente pelo usuário em colunas que não são PK.
6. **Busca por Índice Composto:** combinação de índices primários.

Algoritmos de JOIN

Nested Join

- Um loop dentro de outro.
- Para cada registro em “A” procura-se a correspondência em “B”.
- Ideal para tabelas pequenas ou quando há índices eficientes.

Hash Join

- Cria uma **tabela de dispersão** (hash) em memória com a **tabela menor** e varre a maior buscando correspondências.
- Cria uma tabela menor dos registros “A” e depois busca correspondência em “B”.
- Ideal para grandes volumes de dados desordenados.

Merge Join

- Une duas tabelas grandes de maneira eficiente, desde que ambas estejam ordenadas.
- Ideal para tabelas grandes ordenadas ou quando a consulta exigir ORDER BY na mesma coluna do JOIN.

Views e Segurança

Conceito	Função
View	Abstrair consultas complexas e restringir o acesso a dados de colunas ou de linhas. Esconde colunas sensíveis.
Materialized View	Acelerar consultas analíticas/agregadas aceitando defasagem controlada de dados.
Role/Group	Agrupar e centralizar conjuntos de permissões (GRANT/REVOKE).
User	Identificar e autenticar uma pessoa física ou um sistema no SGBD.

Extra

Qual comando mostra o plano de execução sem normalmente executá-lo?

EXPLAIN. Exibe e gera o plano de execução do otimizador (planner) sem rodar.

Qual comando executa o plano de execução?

EXPLAIN ANALYZE. Executa a consulta no banco de dados para medir os tempos reais de execução e as linhas processadas em cada etapa do plano.